

International Cybersecurity and Digital Forensics Academy (ICDFA)

Basic Computer Skills for Digital Forensics

Lab 3 - Steganography and Hidden Data Detection

Student Name: Kafayat Omolara Animashawun

Student ID: 2025/FWSD/11317

Instructor Name: Aminu Idris

Date of Submission: 05/09/2026

EXECUTIVE SUMMARY

This practical laboratory exercise focused on the identification, creation, extraction, and verification of hidden data using digital steganography techniques. The exercise was conducted within a controlled forensic laboratory environment using a bitmap image as the carrier file and controlled evidence notes as the data to be concealed.

The practical involved preserving and hashing an original carrier image, creating a controlled evidence file, embedding the evidence into the image using Steghide, extracting the concealed data, and verifying its integrity using SHA-256 hashing and file comparison. Additional analysis was performed using xxd and strings to compare the original and stego images and assess whether the hidden information was readily visible through basic inspection.

A controlled password-recovery exercise was also performed using StegSeek and a small wordlist. An independent steganography exercise was subsequently completed using a separate evidence note and passphrase. In both exercises, the recovered evidence produced a matching SHA-256 hash and identical file comparison against the original evidence, demonstrating successful and verifiable recovery.

The exercise demonstrated that steganography can conceal information within an apparently ordinary image without necessarily producing obvious visual or textual indicators of the concealed content. It also demonstrated the importance of hashing, documentation, controlled extraction, and integrity verification in digital forensic investigations.

PURPOSE OF THE ASSIGNMENT

The purpose of this assignment was to provide practical experience in the forensic examination of steganographic content and hidden data. The exercise was designed to demonstrate how information can be concealed within a digital image and how a forensic examiner can identify, extract, recover, and verify such information using appropriate forensic tools.

The assignment also provided an opportunity to apply fundamental digital forensic principles, including evidence preservation, cryptographic hashing, controlled analysis, repeatable examination procedures, and verification of recovered evidence.

OBJECTIVES OF THE ASSIGNMENT

The objectives of the assignment were to:

1. Understand the fundamental concept of digital steganography and how data can be concealed within a carrier file.
2. Prepare and preserve an original carrier image for forensic examination.
3. Generate a controlled evidence note for use during the laboratory exercise.
4. Calculate and document SHA-256 hashes of relevant evidence files before and after processing.
5. Use Steghide to embed controlled evidence into a bitmap image using a password.
6. Extract the concealed evidence from the stego image using the correct password.
7. Verify the integrity of the extracted evidence by comparing its SHA-256 hash with that of the original evidence.
8. Compare the original carrier image and the resulting stego image using file properties, hashing, hexadecimal inspection, and string analysis.
9. Use StegSeek to conduct a controlled password-recovery exercise against the stego image.
10. Verify that evidence recovered through StegSeek is identical to the original concealed evidence.
11. Perform an independent steganography exercise using a separate evidence note, password, and wordlist.
12. Apply forensic documentation and evidence-integrity principles throughout the examination process.

METHODOLOGY OF THE ASSIGNMENT

The practical exercise followed a structured forensic workflow consisting of preparation, evidence preservation, data concealment, extraction, analysis, password recovery, and integrity verification.

First, a dedicated laboratory workspace was prepared with separate directories for the original evidence, working files, extracted evidence, and hash evidence. The original bitmap carrier image was preserved in its original state and its SHA-256 hash was calculated and documented before any steganographic operation was performed.

A controlled evidence file named secret.txt was then created for the exercise. Its SHA-256 hash was calculated and recorded to establish a reference value for subsequent integrity verification.

Steghide was used to embed the controlled evidence file into the original bitmap image. The resulting stego image was saved as a separate working file, ensuring that the original carrier remained unchanged. The resulting stego image was then hashed and its file properties were examined.

The concealed evidence was subsequently extracted from the stego image using Steghide and the correct password. The extracted file was examined and compared with the original evidence. SHA-256 hashes were calculated for both files, while the diff utility was used to confirm that their contents were identical.

Further examination was conducted by comparing the original carrier image and the stego image. Their file sizes and bitmap properties were reviewed, their SHA-256 hashes were compared, and the first 64 bytes were examined using xxd. The strings utility was also used to identify whether obvious readable information could be observed within either file.

A controlled password-recovery exercise was then conducted using StegSeek and a laboratory wordlist containing the known test password. After the password was successfully recovered, the concealed evidence was extracted and its SHA-256 hash was compared against the original evidence.

Finally, an independent exercise was performed using a separate evidence note, a different passphrase, and a separate wordlist. The evidence was embedded into a new stego image, extracted using the correct passphrase, and subsequently recovered using StegSeek. Hash comparison and file comparison were again performed to verify the integrity and consistency of the recovered evidence.

Throughout the exercise, the original carrier and evidence files were retained separately from derived working and extracted files to support evidence preservation and traceability.

TOOLS AND RESOURCES USED

The following tools and resources were used during the practical exercise:

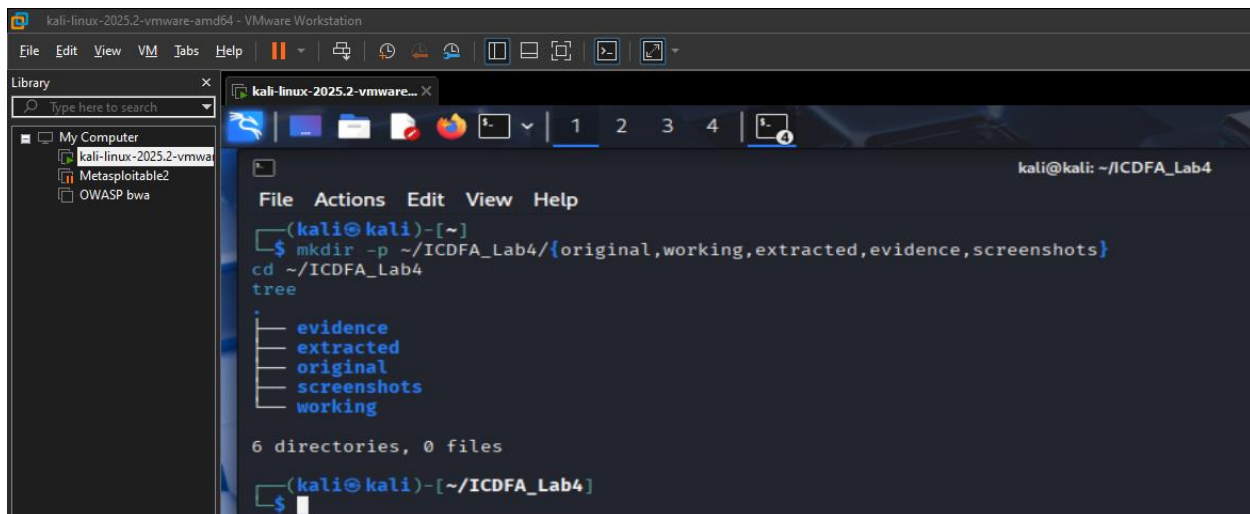
Tool / Resource	Purpose
Kali Linux	Forensic laboratory operating environment used to perform the practical exercises
Steghide 0.6.0	Used to embed and extract concealed data within the bitmap carrier image

Tool / Resource	Purpose
StegSeek 0.6	Used to perform controlled password recovery and extract concealed data
SHA-256 (sha256sum)	Used to establish and verify the cryptographic integrity of evidence files
diff	Used to determine whether original and recovered evidence files were identical
xxd	Used to perform hexadecimal examination and compare file headers/data
strings	Used to identify printable character sequences within the image files
BMP carrier image	Original digital image used as the carrier for the concealed evidence
Controlled evidence notes	Files created specifically for the laboratory steganography exercises
Controlled wordlists	Used to demonstrate password-recovery testing with StegSeek
Git/GitHub	Used to maintain and document the laboratory artifacts and evidence files
SHA-256 hash records	Stored as supporting evidence for integrity verification

The primary forensic resources consisted of the original carrier image, the controlled evidence files, the generated stego images, extracted evidence, password-recovery outputs, wordlists, and corresponding hash records.

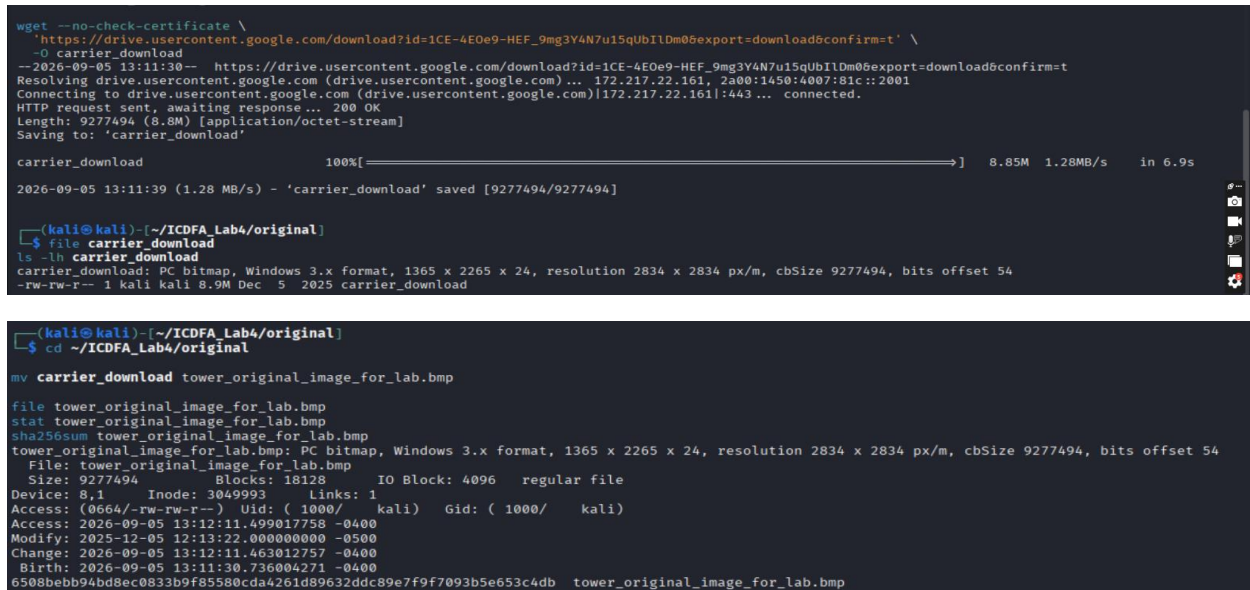
The forensic working environment was prepared using a dedicated directory structure for the ICDFA SBT-DF202 Lab 4 exercise. Separate folders were created for the original carrier image, working files, extracted evidence, supporting evidence records, and screenshots. This structure was established to maintain clear separation between the original evidence and files generated during the examination process.

Evidence of environment preparation



The authorised carrier image obtained for the laboratory exercise was preserved in the original directory and renamed to `tower_original_image_for_lab.bmp` without modifying its contents. The `file` command confirmed that the evidence is a valid 24-bit Windows BMP image with dimensions of 1365×2265 pixels. The `stat` command was used to record the file's size and filesystem metadata, while SHA-256 hashing established a cryptographic baseline for subsequent integrity verification. The calculated SHA-256 hash was `6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db`.

Evidence of the image download



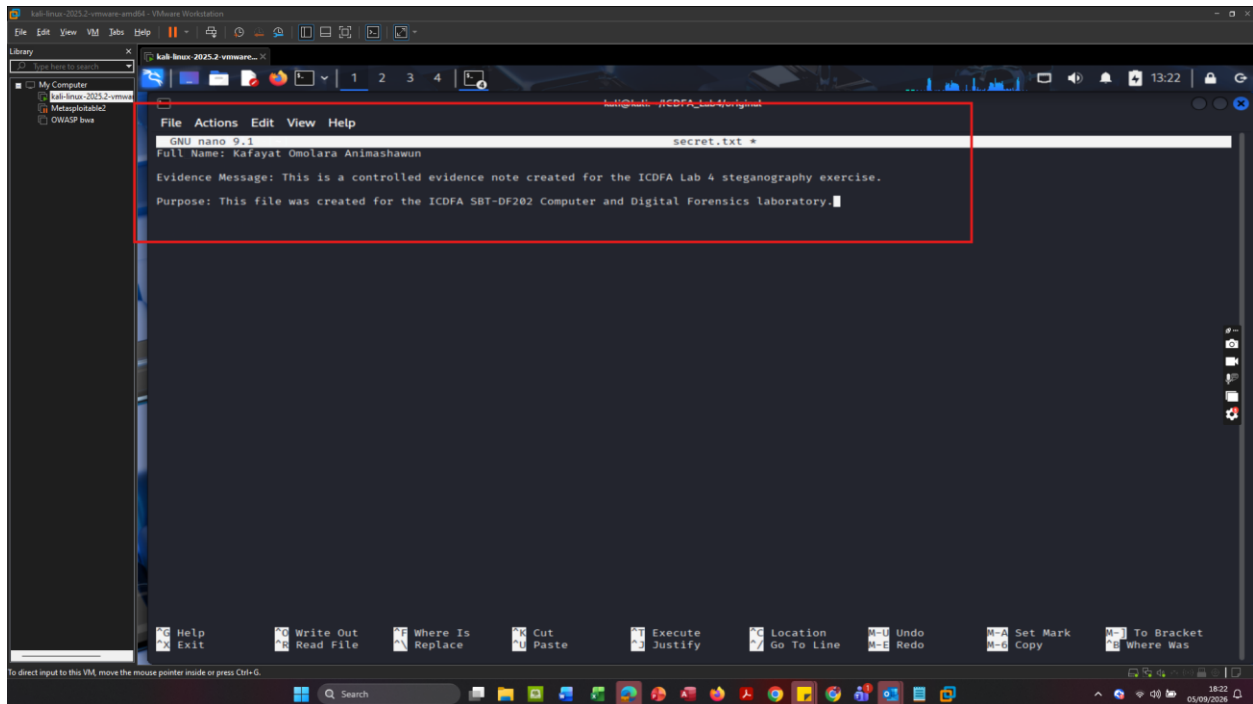
```

mv carrier_download tower_original_image_for_lab.bmp

file tower_original_image_for_lab.bmp
stat tower_original_image_for_lab.bmp
sha256sum tower_original_image_for_lab.bmp
tower_original_image_for_lab.bmp: PC bitmap, Windows 3.x format, 1365 x 2265 x 24, resolution 2834 x 2834 px/m, cbSize 9277494, bits offset 54
  File: tower_original_image_for_lab.bmp
  Size: 9277494      Blocks: 18128      IO Block: 4096      regular file
Device: 8,1      Inode: 3049993      Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/   kali)   Gid: ( 1000/   kali)
Access: 2026-09-05 13:12:11.499017758 -0400
Modify: 2025-12-05 12:13:22.000000000 -0500
Change: 2026-09-05 13:12:11.463012757 -0400
Birth: 2026-09-05 13:11:30.736004271 -0400
6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db tower_original_image_for_lab.bmp

(kali@kali)-[~/ICDFA_Lab4/original]
$ sha256sum tower_original_image_for_lab.bmp | tee ~/ICDFA_Lab4/evidence/original_carrier_sha256.txt
6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db tower_original_image_for_lab.bmp
(kali@kali)-[~/ICDFA_Lab4/original]
$

```



Secret Evidence File Creation

A controlled evidence note was created as the payload for the steganography exercise. The text file contains the investigator's identification, a controlled evidence message, and confirmation that the file was created specifically for the ICDFA SBT-DF202 digital forensics laboratory. Creating the payload separately from the original carrier ensures that the original image remains preserved and available for comparison throughout the examination.

```

(kali@kali)-[~/ICDFA_Lab4/working]
$ cat secret.txt
Full Name: Kafayat Omolara Animashawun

Evidence Message: This is a controlled evidence note created for the ICDFA Lab 4 steganography exercise.

Purpose: This file was created for the ICDFA SBT-DF202 Computer and Digital Forensics Laboratory.
(kali@kali)-[~/ICDFA_Lab4/working]
$

```

The screenshot shows a Windows 10 virtual machine environment. A terminal window titled 'kali@kali-ICDFA_Lab4/working' is open, displaying the contents of a file named 'secret.txt'. The text in the file reads:

```

Full Name: Kafayat Omolara Animashawun

Evidence Message: This is a controlled evidence note created for the ICDFA Lab 4 steganography exercise.

Purpose: This file was created for the ICDFA SBT-DF202 Computer and Digital Forensics Laboratory.

```

The terminal window is overlaid on a desktop background with various application icons. The taskbar at the bottom shows the date as 05/09/2026 and the time as 18:24.

Secret File Hash

A SHA-256 hash was calculated for the original secret.txt payload and preserved in the laboratory evidence directory. This hash provides an integrity reference that can later be compared with the hash of the extracted file. A matching hash after extraction will demonstrate that the concealed payload was recovered without alteration.

```
(kali@kali)-[~/ICDFA_Lab4/working]
└─$ cat secret.txt
Full Name: Kafayat Omolara Animashawun

Evidence Message: This is a controlled evidence note created for the ICDFA Lab 4 steganography exercise.

Purpose: This file was created for the ICDFA SBT-DF202 Computer and Digital Forensics laboratory.

(kali@kali)-[~/ICDFA_Lab4/working]
└─$ sha256sum secret.txt | tee ~/ICDFA_Lab4/evidence/secret_sha256.txt
30a303f20e00e2f61555541a0446cb1685cceb4f4e36b3693c5182dae59dece secret.txt

(kali@kali)-[~/ICDFA_Lab4/working]
└─$
```

Part C: Embed secret.txt into a NEW BMP

Steghide Embedding of Controlled Evidence File.

The steghide embed command was used to conceal the controlled evidence file secret.txt within the authorised original BMP carrier image. The original carrier was referenced from the original directory and was not overwritten; instead, Steghide generated a separate output file named tower_stego_lab4.bmp in the working directory. The password specified for this controlled laboratory exercise was 1234. The terminal output confirms that the embedding operation completed successfully and that the stego image was written successfully.

```
md64 - VMware Workstation
Help  [Icons] [Tools] [View] [Window] [Help]

kali-linux-2025.2-vmware...
┌───┴───┐
File Actions Edit View Help
└───┴───┘

Unpacking steghide (0.5.1+git20240220-2) ...
Setting up libmhash2:amd64 (0.9.9-11+b1) ...
Setting up libmbedtls:amd64 (2.5.8-8+b2) ...
Setting up steghide (0.5.1+git20240220-2) ...
Processing triggers for libc-bin (2.41-9) ...
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for kali-menu (2025.3.0) ...
Scanning processes ...
Scanning linux images ...

Running kernel seems to be up-to-date.
No services need to be restarted.
No containers need to be restarted.
No user sessions are running outdated binaries.
No VM guests are running outdated hypervisor (qemu) binaries on this host.

(kali@kali)-[~/ICDFA_Lab4/working]
└─$ steghide --version
steghide version 0.6.0

(kali@kali)-[~/ICDFA_Lab4/working]
└─$ steghide embed \
    -ef secret.txt \
    -cf ../original/tower_original_image_for_lab.bmp \
    -sf tower_stego_lab4.bmp \
    -p 1234
embedding "secret.txt" in "../original/tower_original_image_for_lab.bmp" ... done
writing stego file "tower_stego_lab4.bmp" ... done

(kali@kali)-[~/ICDFA_Lab4/working]
└─$ file tower_stego_lab4.bmp
ls -lh tower_stego_lab4.bmp
sha256sum tower_stego_lab4.bmp | tee ~/ICDFA_Lab4/evidence/stego_image_sha256.txt
tower_stego_lab4.bmp: PC bitmap, Windows 3.x format, 1365 x 2265 x 24, resolution 2834 x 2834 px/m, cbSize 9277494, bits offset 54
-rw-rw-r-- 1 kali kali 8.9M Sep  5 13:31 tower_stego_lab4.bmp
edcb2d6b27465546ddc0c303fc4f362f26da13fa0974b518bcec17af48936dd2 tower_stego_lab4.bmp

(kali@kali)-[~/ICDFA_Lab4/working]
└─$
```

Verification and SHA-256 Hashing of Stego Image.

Following the successful embedding operation, the resulting tower_stego_lab4.bmp file was examined using the file command to confirm that it remained a valid 24-bit Windows BMP with the same 1365 × 2265 dimensions as the original carrier. The file size was also examined using ls -lh. A SHA-256 hash was subsequently calculated to establish a cryptographic identifier for the resulting stego image. The recorded SHA-256 hash is edcb2d6b27465546ddc0c303fc4f362f26da13fa0974b518bcec17af48936dd2. This hash will later be compared with the original carrier hash to demonstrate that the image contents changed despite retaining the same apparent file characteristics.

```
(kali@kali)-[~/ICDFA_Lab4/working]
$ file tower_stego_lab4.bmp
ls -lh tower_stego_lab4.bmp
sha256sum tower_stego_lab4.bmp | tee ~/ICDFA_Lab4/evidence/stego_image_sha256.txt
tower_stego_lab4.bmp: PC bitmap, Windows 3.x format, 1365 x 2265 x 24, resolution 2834 x 2834 px/m, cbSize 9277494, bits offset 54
-rw-rw-r-- 1 kali kali 8.9M Sep  5 13:31 tower_stego_lab4.bmp
edcb2d6b27465546ddc0c303fc4f362f26da13fa0974b518bcec17af48936dd2  tower_stego_lab4.bmp
```

The Steghide operation worked exactly as expected, and the recorded result include:

Evidence	Result
Steghide	0.6.0
Carrier	tower_original_image_for_lab.bmp
Hidden file	secret.txt
Password	1234
Stego output	tower_stego_lab4.bmp
Stego format	24-bit BMP, 1365 × 2265
Stego SHA-256	edcb2d6b27465546ddc0c303fc4f362f26da13fa0974b518bcec17af48936dd2

The fact that the stego file reports the same dimensions and file size as the original is also useful evidence. The contents have changed, however, as the SHA-256 hashes will demonstrate.

Successful Extraction of Concealed Evidence File.

The stego image tower_stego_lab4.bmp was examined using Steghide with the correct laboratory password, 1234. The concealed evidence file was successfully extracted and written to the designated extracted directory as extracted_secret.txt. The successful extraction confirms that data had been embedded within the carrier image and could be recovered using the appropriate extraction tool and password without modifying the preserved original carrier image.

```

L-$ steghide embed \
  -ef secret.txt \
  -cf ../original/tower_original_image_for_lab.bmp \
  -sf tower_stego_lab4.bmp \
  -p 1234
embedding "secret.txt" in "../original/tower_original_image_for_lab.bmp" ... done
writing stego file "tower_stego_lab4.bmp" ... done

(kali@kali)-[~/ICDFA_Lab4/working]
$ file tower_stego_lab4.bmp
ls -lh tower_stego_lab4.bmp
sha256sum tower_stego_lab4.bmp | tee ~/ICDFA_Lab4/evidence/stego_image_sha256.txt
tower_stego_lab4.bmp: PC bitmap, Windows 3.x format, 1365 x 2265 x 24, resolution 2834 x 2834 px/m, cbSize 9277494, bits offset 54
-rw-rw-r-- 1 kali kali 8.9M Sep  5 13:31 tower_stego_lab4.bmp
edcb2d6b27465546ddc0c303fc4f362f26da13fa0974b518bcec17af48936dd2  tower_stego_lab4.bmp

(kali@kali)-[~/ICDFA_Lab4/working]
$ mkdir -p ~/ICDFA_Lab4/extracted

(kali@kali)-[~/ICDFA_Lab4/working]
$ steghide extract \
  -sf tower_stego_lab4.bmp \
  -xf ../extracted/extracted_secret.txt \
  -p 1234
wrote extracted data to "../extracted/extracted_secret.txt".

```

Examination of Recovered Evidence Content.

The contents of the extracted file were displayed to verify that the concealed evidence had been recovered successfully. The recovered text contains the expected full name, controlled evidence message, and statement identifying the file as part of the ICDFA SBT-DF202 Computer and Digital Forensics laboratory exercise. The recovered content corresponds to the information originally placed in secret.txt, providing an initial content-level verification of the extraction process.

```

(kali@kali)-[~/ICDFA_Lab4/working]
$ mkdir -p ~/ICDFA_Lab4/extracted

(kali@kali)-[~/ICDFA_Lab4/working]
$ steghide extract \
  -sf tower_stego_lab4.bmp \
  -xf ../extracted/extracted_secret.txt \
  -p 1234
wrote extracted data to "../extracted/extracted_secret.txt".

(kali@kali)-[~/ICDFA_Lab4/working]
$ cat ../extracted/extracted_secret.txt
Full Name: Kafayat Omolara Animashawun

Evidence Message: This is a controlled evidence note created for the ICDFA Lab 4 steganography exercise.

Purpose: This file was created for the ICDFA SBT-DF202 Computer and Digital Forensics laboratory.

```

Cryptographic Hash Verification of Extracted Evidence.

A SHA-256 hash was calculated for the recovered extracted_secret.txt file to provide a cryptographic integrity value. The resulting hash was 30a303f20e00e2f615555541a0446cb1685cceb4fe36b3693c5182dae59dece. This value is identical to the SHA-256 hash previously calculated for the original secret.txt, indicating that the recovered evidence has the same underlying byte content as the original file.

```

(kali@kali)-[~/ICDFA_Lab4/working]
$ cat ../extracted/extracted_secret.txt
Full Name: Kafayat Omolara Animashawun

Evidence Message: This is a controlled evidence note created for the ICDFA Lab 4 steganography exercise.

Purpose: This file was created for the ICDFA SBT-DF202 Computer and Digital Forensics laboratory.

(kali@kali)-[~/ICDFA_Lab4/working]
$ sha256sum ../extracted/extracted_secret.txt | tee ~/ICDFA_Lab4/evidence/extracted_secret_sha256.txt
30a303f20e00e2f615555541a0446cb1685cceb4fe36b3693c5182dae59dece  ../extracted/extracted_secret.txt

```

Direct Comparison of Original and Extracted Evidence Files.

The Linux diff utility was used to perform a direct comparison between the original secret.txt and the file recovered from the stego image. The command reported that both files were identical. Together with the matching SHA-256 hashes, this provides strong verification that the concealed evidence was extracted completely and without alteration during the Steghide embedding and extraction process.

```
(kali@kali)-[~/ICDFA_Lab4/working]
$ cat ../extracted/extracted_secret.txt
Full Name: Kafayat Omolara Animashawun
Evidence Message: This is a controlled evidence note created for the ICDFA Lab 4 steganography exercise.
Purpose: This file was created for the ICDFA SBT-DF202 Computer and Digital Forensics laboratory.

(kali@kali)-[~/ICDFA_Lab4/working]
$ sha256sum ../extracted/extracted_secret.txt | tee ~/ICDFA_Lab4/evidence/extracted_secret_sha256.txt
30a303f20e00e2f615555541a0446cb1685cceb4fe36b3693c5182dae59dece ../extracted/extracted_secret.txt

(kali@kali)-[~/ICDFA_Lab4/working]
$ diff -s secret.txt ../extracted/extracted_secret.txt
Files secret.txt and ../extracted/extracted_secret.txt are identical
```

Forensic Comparison of Original Carrier and Stego Image.

The original carrier image and the stego image were examined using the file, ls -lh, and sha256sum utilities. Both files were identified as 24-bit Windows BMP images with identical dimensions of 1365 × 2265 pixels and the same reported file size of 9,277,494 bytes. Despite these matching file characteristics, their SHA-256 hashes are different. The original carrier produced 6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db, while the stego image produced edcb2d6b27465546ddc0c303fc4f362f26da13fa0974b518bcec17af48936dd2. The differing cryptographic hashes confirm that the underlying file data was modified during the steganographic embedding process, even though the observable file properties remained unchanged.

```
(kali@kali)-[~/ICDFA_Lab4/working]
$ sha256sum ../extracted/extracted_secret.txt | tee ~/ICDFA_Lab4/evidence/extracted_secret_sha256.txt
30a303f20e00e2f615555541a0446cb1685cceb4fe36b3693c5182dae59dece ../extracted/extracted_secret.txt

(kali@kali)-[~/ICDFA_Lab4/working]
$ diff -s secret.txt ../extracted/extracted_secret.txt
Files secret.txt and ../extracted/extracted_secret.txt are identical

(kali@kali)-[~/ICDFA_Lab4/working]
$ cd ~/ICDFA_Lab4/working

echo "===== ORIGINAL CARRIER ====="
file ../original/tower_original_image_for_lab.bmp
ls -lh ../original/tower_original_image_for_lab.bmp
sha256sum ../original/tower_original_image_for_lab.bmp

echo "===== STEGO IMAGE ====="
file tower_stego_lab4.bmp
ls -lh tower_stego_lab4.bmp
sha256sum tower_stego_lab4.bmp

===== ORIGINAL CARRIER =====
../original/tower_original_image_for_lab.bmp: PC bitmap, Windows 3.x format, 1365 x 2265 x 24, resolution 2834 x 2834 px/m, cbSize 9277494, bits offset 54
-rw-rw-r-- 1 kali kali 8.9M Dec 5 2025 ../original/tower_original_image_for_lab.bmp
6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db ../original/tower_original_image_for_lab.bmp

===== STEGO IMAGE =====
tower_stego_lab4.bmp: PC bitmap, Windows 3.x format, 1365 x 2265 x 24, resolution 2834 x 2834 px/m, cbSize 9277494, bits offset 54
-rw-rw-r-- 1 kali kali 8.9M Sep 5 13:31 tower_stego_lab4.bmp
edcb2d6b27465546ddc0c303fc4f362f26da13fa0974b518bcec17af48936dd2 tower_stego_lab4.bmp

(kali@kali)-[~/ICDFA_Lab4/working]
$
```

Hexadecimal Comparison of Initial File Bytes with xxd

The first 64 bytes of both the original carrier and the stego image were examined using the xxd utility. The hexadecimal output shows that the initial bytes of both files are identical, including the BMP file signature (42 4d) and the associated header information. This indicates that the steganographic modification did not alter the beginning of the BMP file or its basic structural header. The result demonstrates an important forensic characteristic of steganography: a file can retain its normal format and header structure while modifications occur elsewhere in the file's data. Consequently, header inspection alone would not be sufficient to determine whether the image contains concealed information.

```

e4cb280027405940ddc0c5057c47302f20da137a09740510b0cc17a740930002 tower_stego_lab4.bmp

(kali@kali)~[~/ICDFA_Lab4/working]
$ echo "===== ORIGINAL FIRST 64 BYTES ====="
xxd -l 64 ../original/tower_original_image_for_lab.bmp

echo
echo "===== STEGO FIRST 64 BYTES ====="
xxd -l 64 tower_stego_lab4.bmp
===== ORIGINAL FIRST 64 BYTES =====
00000000: 424d 3690 8d00 0000 0000 3600 0000 2800  BM6 .. ...6 ... (.
00000010: 0000 5505 0000 d908 0000 0100 1800 0000  ..U.. .. .. .
00000020: 0000 0000 0000 120b 0000 120b 0000 0000  .... .. .. .
00000030: 0000 0000 0000 1e22 3520 2338 1e20 381f  .... "5 #8. 8.

===== STEGO FIRST 64 BYTES =====
00000000: 424d 3690 8d00 0000 0000 3600 0000 2800  BM6 .. ...6 ... (.
00000010: 0000 5505 0000 d908 0000 0100 1800 0000  ..U.. .. .. .
00000020: 0000 0000 0000 120b 0000 120b 0000 0000  .... .. .. .
00000030: 0000 0000 0000 1e22 3520 2338 1e20 381f  .... "5 #8. 8.

(kali@kali)~[~/ICDFA_Lab4/working]
$

```

Identical file size and structural metadata do not establish file integrity; cryptographic hashing provides a much stronger means of detecting content modification.

Printable String Examination of the Original and Stego Images.

The strings utility was used to perform a basic forensic examination of both the original carrier and the stego image for sequences of printable characters. A minimum length of six characters was specified to reduce very short and potentially insignificant character sequences. The output primarily consists of apparently binary/image-data-derived character sequences rather than an obvious human-readable hidden message. This is consistent with the nature of the BMP files and demonstrates that a conventional printable-string search is not sufficient to identify the concealed evidence. The successful recovery of secret.txt using Steghide provides stronger evidence that the information was embedded using a steganographic mechanism rather than stored as ordinary readable text.


```

(kali@kali)-[~/ICDFA_Lab4/working]
$ stegseek --version
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

(kali@kali)-[~/ICDFA_Lab4/working]
$ cd ~/ICDFA_Lab4/working

printf "1234\n" > lab4_wordlist.txt

cat lab4_wordlist.txt
1234

(kali@kali)-[~/ICDFA_Lab4/working]
$ sha256sum lab4_wordlist.txt | tee ~/ICDFA_Lab4/evidence/lab4_wordlist_sha256.txt
a883dafc480d466ee04e0d6da986bd78eb1fdd2178d04693723da3a8f95d42f4 lab4_wordlist.txt

```

Successful Dictionary-Based Password Recovery Using StegSeek.

StegSeek was used with the controlled laboratory wordlist to test password candidates against the steganographic image. The tool successfully identified the passphrase 1234 and reported the original embedded filename as secret.txt. StegSeek subsequently extracted the concealed data to tower_stego_lab4.bmp.out. This result demonstrates that when the correct password is present in a candidate wordlist, dictionary-based password recovery can successfully recover data protected by a steganographic tool such as Steghide.

```

(kali@kali)-[~/ICDFA_Lab4/working]
$ stegseek tower_stego_lab4.bmp lab4_wordlist.txt
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

[!] Found passphrase: "1234"
[!] Original filename: "secret.txt".
[!] Extracting to "tower_stego_lab4.bmp.out".

(kali@kali)-[~/ICDFA_Lab4/working]
$ cat tower_stego_lab4.bmp.out
Successful Dictionary-Based Password Recovery Using StegSeek. StegSeek was used with the controlled laboratory wordlist to test password candidates against the steganographic image. The tool successfully identified the passphrase 1234 and reported the original embedded filename as secret.txt. StegSeek subsequently extracted the concealed data to tower_stego_lab4.bmp.out. This result demonstrates that when the correct password is present in a candidate wordlist, dictionary-based password recovery can successfully recover data protected by a steganographic tool such as Steghide.

(kali@kali)-[~/ICDFA_Lab4/working]
$ sha256sum tower_stego_lab4.bmp.out secret.txt
30a303f20e00e2f615555541a0446cb1685cceb4fe36b3693c5182dae59dece tower_stego_lab4.bmp.out
30a303f20e00e2f615555541a0446cb1685cceb4fe36b3693c5182dae59dece secret.txt

(kali@kali)-[~/ICDFA_Lab4/working]
$

```

Integrity Verification of Data Recovered by StegSeek.

The file recovered by StegSeek was compared with the original secret.txt using SHA-256 hashing and the diff utility. Both files produced the identical SHA-256 hash, 30a303f20e00e2f615555541a0446cb1685cceb4fe36b3693c5182dae59dece, and the diff command confirmed that the files are identical. This provides cryptographic and content-level verification that StegSeek successfully recovered the concealed evidence without alteration.

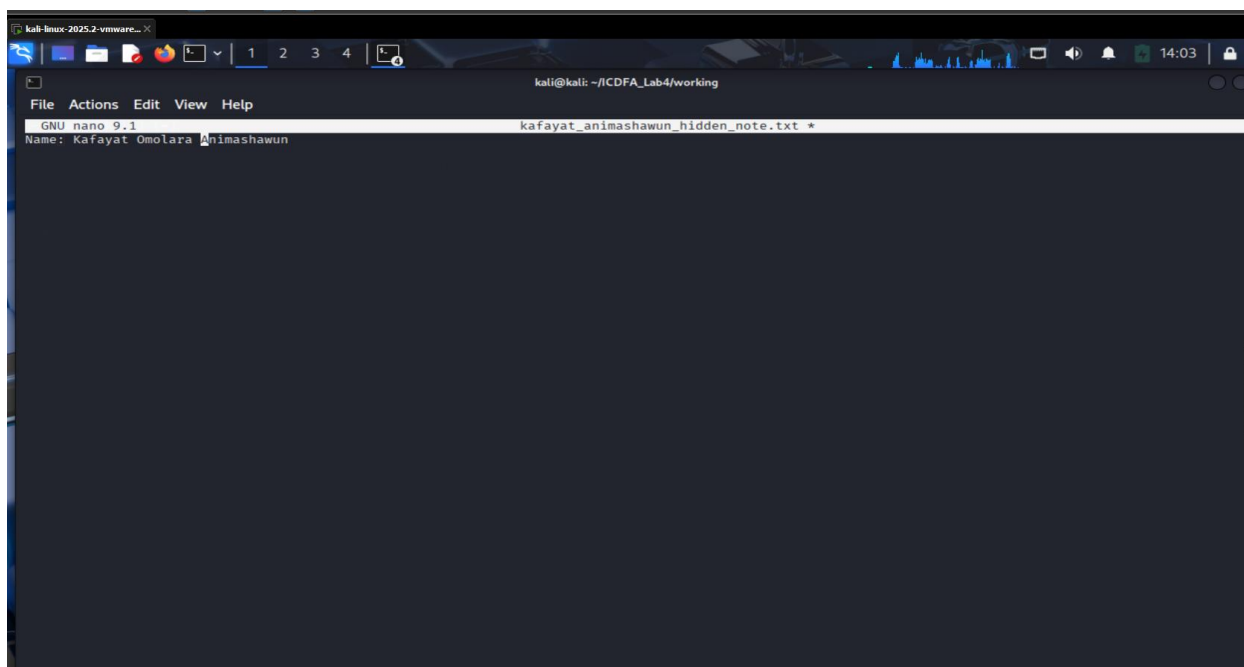
```

(kali@kali)-[~/ICDFA_Lab4/working]
$ sha256sum tower_stego_lab4.bmp.out secret.txt
30a303f20e00e2f615555541a0446cb1685cceb4fe36b3693c5182dae59dece tower_stego_lab4.bmp.out
30a303f20e00e2f615555541a0446cb1685cceb4fe36b3693c5182dae59dece secret.txt

(kali@kali)-[~/ICDFA_Lab4/working]
$ diff -s secret.txt tower_stego_lab4.bmp.out
Files secret.txt and tower_stego_lab4.bmp.out are identical

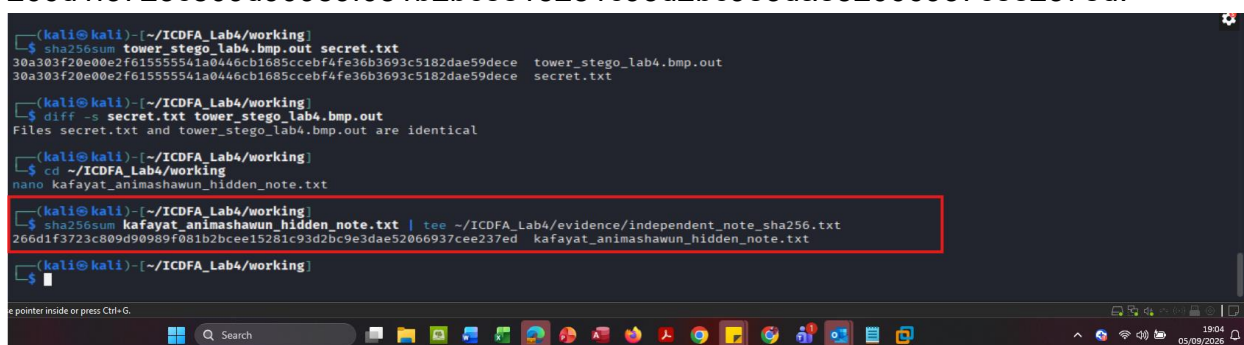
(kali@kali)-[~/ICDFA_Lab4/working]
$

```



Creation and Hashing of the Independent Evidence Note.

An independent evidence note was created for the second part of the practical exercise using the filename `kafayat_animashawun_hidden_note.txt`. The note explains the concept of steganography, distinguishes it from encryption, and describes the forensic relevance of detecting concealed information. A SHA-256 hash was calculated before embedding to establish a reference value for subsequent integrity verification. The resulting hash was `266d1f3723c809d90989f081b2bcee15281c93d2bc9e3dae52066937cee237ed`.



Independent Steganographic Embedding and Hashing.

The independently created evidence note was embedded into a new BMP stego image using Steghide and the password `Forensics@2026`. The original carrier image was used as the carrier to ensure that the independent experiment was separate from the earlier controlled demonstration. Steghide successfully completed the embedding operation, and the resulting file retained the expected BMP format, dimensions, and file size. A SHA-

256 hash of the independent stego image was calculated as 4bb04ef724409df9e2f620f2fdbee1fcf7f1d6a9532d3009e11b9c2da9f8ca1c for subsequent integrity verification.

```
(kali@kali)-[~/ICDFA_Lab4/working]
$ steghide embed \
  -ef kafayat_animashawun_hidden_note.txt \
  -sf ../original/tower_original_image_for_lab.bmp \
  -sf kafayat_animashawun_stego.bmp \
  -p 'Forensics@2026'
embedding "kafayat_animashawun_hidden_note.txt" in "../original/tower_original_image_for_lab.bmp" ... done
writing stego file "kafayat_animashawun_stego.bmp" ... done

(kali@kali)-[~/ICDFA_Lab4/working]
$ file kafayat_animashawun_stego.bmp
sha256sum kafayat_animashawun_stego.bmp | tee ~/ICDFA_Lab4/evidence/independent_stego_sha256.txt
kafayat_animashawun_stego.bmp: PC bitmap, Windows 3.X format, 1365 x 2265 x 24, resolution 2834 x 2834 px/m, cbSize 9277494, bits offset 54
4bb04ef724409df9e2f620f2fdbee1fcf7f1d6a9532d3009e11b9c2da9f8ca1c kafayat_animashawun_stego.bmp
```

Successful Extraction of the Independent Hidden Note.

The independently created steganographic image was processed using Steghide with the correct passphrase Forensics@2026. The tool successfully extracted the concealed file to the designated evidence extraction directory. The recovered content was then displayed using the cat command, confirming that the concealed note was successfully retrieved from the stego image.

```
(kali@kali)-[~/ICDFA_Lab4/working]
$ steghide extract \
  -sf kafayat_animashawun_stego.bmp \
  -xf ../extracted/kafayat_animashawun_extracted.txt \
  -p 'Forensics@2026'
wrote extracted data to "../extracted/kafayat_animashawun_extracted.txt".

(kali@kali)-[~/ICDFA_Lab4/working]
$ cat ../extracted/kafayat_animashawun_extracted.txt
Name: Kafayat Omolara Animashawun

(kali@kali)-[~/ICDFA_Lab4/working]
$ diff -s kafayat_animashawun_hidden_note.txt ../extracted/kafayat_animashawun_extracted.txt
Files kafayat_animashawun_hidden_note.txt and ../extracted/kafayat_animashawun_extracted.txt are identical
```

Integrity Verification of the Independently Recovered Note.

The original independent note and the file recovered from the stego image were compared using SHA-256 hashing and the diff utility. The diff result confirmed that both files contain identical content. Matching cryptographic hashes would additionally establish that the recovered evidence is byte-for-byte identical to the original note.

```
(kali@kali)-[~/ICDFA_Lab4/working]
$ steghide extract \
  -sf kafayat_animashawun_stego.bmp \
  -xf ../extracted/kafayat_animashawun_extracted.txt \
  -p 'Forensics@2026'
wrote extracted data to "../extracted/kafayat_animashawun_extracted.txt".

(kali@kali)-[~/ICDFA_Lab4/working]
$ cat ../extracted/kafayat_animashawun_extracted.txt
Name: Kafayat Omolara Animashawun

(kali@kali)-[~/ICDFA_Lab4/working]
$ diff -s kafayat_animashawun_hidden_note.txt ../extracted/kafayat_animashawun_extracted.txt
Files kafayat_animashawun_hidden_note.txt and ../extracted/kafayat_animashawun_extracted.txt are identical
```

Cryptographic Integrity Verification of the Independent Extraction.

The original independent note and the file recovered from the steganographic image were compared using SHA-256 hashing and the diff utility. Both files produced the identical SHA-256 hash,

266d1f3723c809d90989f081b2bcee15281c93d2bc9e3dae52066937cee237ed. The diff command also confirmed that the files are identical. These results establish that the independently embedded note was successfully recovered without modification.

```
(kali@kali)-[~/ICDFA_Lab4/working]
$ diff -s kafayat_animashawun_hidden_note.txt ../extracted/kafayat_animashawun_extracted.txt
Files kafayat_animashawun_hidden_note.txt and ../extracted/kafayat_animashawun_extracted.txt are identical

(kali@kali)-[~/ICDFA_Lab4/working]
$ sha256sum kafayat_animashawun_hidden_note.txt ../extracted/kafayat_animashawun_extracted.txt
266d1f3723c809d90989f081b2bcee15281c93d2bc9e3dae52066937cee237ed kafayat_animashawun_hidden_note.txt
266d1f3723c809d90989f081b2bcee15281c93d2bc9e3dae52066937cee237ed ../extracted/kafayat_animashawun_extracted.txt

(kali@kali)-[~/ICDFA_Lab4/working]
$
```

Controlled Wordlist Creation for Independent Password-Recovery Testing.

A dedicated wordlist containing the password selected for the independent steganography experiment, Forensics@2026, was created. The wordlist was hashed using SHA-256, producing 241f2c73c03fb2e8db423a9ccc2b09947ba4ec19da77341e27fcdcf9a87ebdd. Recording the hash documents the exact password-testing input used during the experiment and supports reproducibility.

```
(kali@kali)-[~/ICDFA_Lab4/working]
$ printf 'Forensics@2026\n' > independent_wordlist.txt
cat independent_wordlist.txt
sha256sum independent_wordlist.txt | tee ~/ICDFA_Lab4/evidence/independent_wordlist_sha256.txt
Forensics@2026
241f2c73c03fb2e8db423a9ccc2b09947ba4ec19da77341e27fcdcf9a87ebdd independent_wordlist.txt

(kali@kali)-[~/ICDFA_Lab4/working]
$ stegseek kafayat_animashawun_stego.bmp independent_wordlist.txt
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

[i] Found passphrase: "Forensics@2026"
[i] Original filename: "kafayat_animashawun_hidden_note.txt".
[i] Extracting to "kafayat_animashawun_stego.bmp.out".
```

Successful Password Recovery Against the Independent Stego Image.

StegSeek was executed against the independently created stego image using the controlled wordlist. The tool successfully identified the passphrase Forensics@2026, recovered the original embedded filename kafayat_animashawun_hidden_note.txt, and extracted the concealed data to kafayat_animashawun_stego.bmp.out. This demonstrates that dictionary-based password testing can recover steganographically concealed data when the correct passphrase is present in the candidate wordlist.

```
(kali@kali)-[~/ICDFA_Lab4/working]
$ printf 'Forensics@2026\n' > independent_wordlist.txt
cat independent_wordlist.txt
sha256sum independent_wordlist.txt | tee ~/ICDFA_Lab4/evidence/independent_wordlist_sha256.txt
Forensics@2026
241f2c73c03fb2e8db423a9ccc2b09947ba4ec19da77341e27fcdcf9a87ebdd independent_wordlist.txt

(kali@kali)-[~/ICDFA_Lab4/working]
$ stegseek kafayat_animashawun_stego.bmp independent_wordlist.txt
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

[i] Found passphrase: "Forensics@2026"
[i] Original filename: "kafayat_animashawun_hidden_note.txt".
[i] Extracting to "kafayat_animashawun_stego.bmp.out".
```

The evidence shows:

- Wordlist: independent_wordlist.txt

- Password candidate: Forensics@2026
- Wordlist SHA-256:
241f2c73c03fb2e8db423a9ccc2b09947ba4ec19da77341e27fdcdcf9a87ebdd
- StegSeek found the correct passphrase.
- It recovered the original filename: kafayat_animashawun_hidden_note.txt
- Extraction output: kafayat_animashawun_stego.bmp.out

Integrity Verification of Data Recovered Through StegSeek.

The file recovered by StegSeek from the independently created stego image was compared with the original hidden note using SHA-256 hashing and the diff utility. Both files produced the identical SHA-256 hash, 266d1f3723c809d90989f081b2bcee15281c93d2bc9e3dae52066937cee237ed, while diff confirmed that the files are identical. These results demonstrate that the correct password was successfully used to recover the concealed evidence and that the recovered file is byte-for-byte identical to the original note.

```
(kali@kali)-[~/ICDFA_Lab4/working]
$ sha256sum kafayat_animashawun_hidden_note.txt kafayat_animashawun_stego.bmp.out
266d1f3723c809d90989f081b2bcee15281c93d2bc9e3dae52066937cee237ed  kafayat_animashawun_hidden_note.txt
266d1f3723c809d90989f081b2bcee15281c93d2bc9e3dae52066937cee237ed  kafayat_animashawun_stego.bmp.out

(kali@kali)-[~/ICDFA_Lab4/working]
$ diff -s kafayat_animashawun_hidden_note.txt kafayat_animashawun_stego.bmp.out
Files kafayat_animashawun_hidden_note.txt and kafayat_animashawun_stego.bmp.out are identical
```

```

kali@kali: ~/ICDFA_Lab4
File Actions Edit View Help
(kali@kali)-[~/ICDFA_Lab4/working]
$ Files kafayat_animashawun_hidden_note.txt and kafayat_animashawun_stego.bmp.out are identical
Files: command not found

(kali@kali)-[~/ICDFA_Lab4/working]
$ cd ~/ICDFA_Lab4

echo "==== EVIDENCE FILES ===="
find evidence -maxdepth 1 -type f -printf '%f\n' | sort

echo
echo "==== WORKING FILES ===="
find working -maxdepth 1 -type f -printf '%f\n' | sort

echo
echo "==== EXTRACTED FILES ===="
find extracted -maxdepth 1 -type f -printf '%f\n' | sort
EVIDENCE FILES
extracted_secret_sha256.txt
independent_note_sha256.txt
independent_stego_sha256.txt
independent_wordlist_sha256.txt
lab4_wordlist_sha256.txt
original_carrier_sha256.txt
secret_sha256.txt
stego_image_sha256.txt

==== WORKING FILES ====
independent_wordlist.txt
kafayat_animashawun_hidden_note.txt
kafayat_animashawun_stego.bmp
kafayat_animashawun_stego.bmp.out
lab4_wordlist.txt
secret.txt
tower_stego_lab4.bmp
tower_stego_lab4.bmp.out

==== EXTRACTED FILES ====
extracted_secret.txt
kafayat_animashawun_extracted.txt

(kali@kali)-[~/ICDFA_Lab4]
$

```

CONCLUSION

The practical exercise successfully demonstrated the principles and forensic implications of digital steganography and hidden data detection. A controlled evidence note was successfully embedded into a bitmap carrier image using Steghide and subsequently extracted using the correct password. The successful recovery was confirmed through SHA-256 hash comparison and file comparison, establishing that the extracted evidence was identical to the original evidence.

The comparison between the original carrier image and the stego image demonstrated that basic file properties alone may not reveal the presence of concealed information. Although the original and stego images retained the same file size and bitmap characteristics, their SHA-256 hashes were different, confirming that the files were not identical. Hexadecimal and string-based examination also demonstrated the limitations of basic inspection techniques when attempting to identify hidden content.

The controlled StegSeek exercise further demonstrated how weak or predictable passwords can be recovered through dictionary-based password testing. The laboratory password was successfully identified using a controlled wordlist, after which the concealed evidence was extracted and its integrity verified. The independent exercise reinforced these findings by demonstrating successful embedding, extraction, password recovery, and integrity verification using a different evidence note, passphrase, and wordlist.

Overall, the laboratory exercise highlighted the importance of combining multiple forensic techniques rather than relying on a single detection method. Cryptographic hashing provided reliable evidence-integrity verification, while Steghide and StegSeek demonstrated practical methods for concealing, extracting, and recovering hidden information. The exercise also emphasized the importance of preserving original evidence, maintaining separate working copies, documenting forensic procedures, and verifying recovered data.

The practical knowledge gained from this exercise provides a foundation for identifying and investigating potential steganographic activity during digital forensic examinations, particularly where malicious actors may attempt to conceal information within seemingly legitimate digital files.